Proposta de trabalho prático estruturada em formato de **requisitos de projeto (User Stories / Tasks)**. Essa abordagem simula o dia a dia de um desenvolvedor aplicando os conceitos de Programação Orientada a Objetos (POO) na prática.
O tema escolhido para o trabalho é o **Desenvolvimento de um Sistema de Gestão de Frota de Veículos e Entregas**.

Trabalho Prático: Sistema de Gestão de Frota e Logística

Contexto Geral

Sua equipe foi contratada para desenvolver o back-end de um sistema de logística. O sistema deve gerenciar diferentes tipos de veículos, motoristas, rotas e custos de viagem, garantindo segurança nos dados e resiliência a falhas.

Requisitos do Projeto

Requisito 1: Abstração e Classes (O Core do Sistema)

- **Tarefa**: Crie a estrutura base para representar um veículo genérico no sistema.
- **Critérios de Aceite**:
  - Implemente uma classe abstrata chamada `Veiculo`.
  - Esta classe não deve ser instanciada diretamente.
  - Defina as propriedades comuns: `placa`, `modelo` e `capacidadeCarga`.
  - Defina um método abstrato chamado `calcularAutonomia()`.

Requisito 2: Encapsulamento (Proteção de Dados)

- **Tarefa**: Garanta que os dados críticos dos veículos não sejam modificados indevidamente.
- **Critérios de Aceite**:
  - Atributos como `placa` e `capacidadeCarga` devem ser estritamente privados (`private`).
  - Crie métodos de leitura (`getters`) para todos os atributos.
  - Crie métodos de modificação (`setters`) apenas onde fizer sentido. O setter de `capacidadeCarga` deve impedir valores menores ou iguais a zero.

Requisito 3: Objetos e Instanciação (Simulação da Frota)

- **Tarefa**: Crie o ambiente de teste (classe `Main` ou equivalente) para rodar o sistema.
- **Critérios de Aceite**:
  - Instancie objetos reais a partir das subclasses criadas nos próximos passos.

    - Exiba no console os dados formatados de pelo menos 3 veículos cadastrados na frota.

Requisito 4: Herança (Especialização de Veículos)

- **Tarefa**: O sistema precisa suportar dois tipos específicos de transporte: Caminhões e Vans.
- **Critérios de Aceite**:
    - Crie a classe `Caminhao` herdando de `Veiculo`. Adicione o atributo privado `quantidadeEixos`.
    - Crie a classe `Van` herdando de `Veiculo`. Adicione o atributo privado `possuiRefrigeracao` (booleano).

Requisito 5: Polimorfismo e Sobrescrita (Cálculos Específicos)

- **Tarefa**: Cada tipo de veículo consome combustível e calcula sua autonomia de forma diferente devido ao peso e eixos.
- **Critérios de Aceite**:
    - Sobrescreva o método `calcularAutonomia()` na classe `Caminhao`. (Ex: autonomia padrão reduzida em 6% para cada eixo adicional).
    - Sobrescreva o método `calcularAutonomia()` na classe `Van`. (Ex: se tiver refrigeração ligada, o consumo aumenta em 10%).

Requisito 6: Sobrecarga (Flexibilidade nos Métodos)

- **Tarefa**: Permitir o cálculo do custo estimado da viagem de duas formas diferentes.
- **Critérios de Aceite**:
    - Na classe `Veiculo`, crie um método chamado `calcularCustoViagem(double distancia)`. Ele calcula o custo baseado apenas no preço padrão do combustível.
    - Crie uma sobrecarga: `calcularCustoViagem(double distancia, double taxaPedagio)`. Ele deve somar o valor do pedágio ao custo final da rota.

Requisito 7: Interfaces (Padronização de Serviços)

- **Tarefa**: Alguns veículos precisam passar por vistorias e manutenções obrigatórias regulamentadas por lei.
- **Critérios de Aceite**:
    - Crie uma interface chamada `Manutenivel`.
    - Defina os métodos `realizarRevisao()` e `precisaManutencao()`.
    - Faça apenas a classe `Caminhao` implementar essa interface (pois vistorias de caminhões são mais rígidas).

Requisito 8: Tratamento de Exceções (Robustez do Sistema)

- **Tarefa**: O sistema não pode quebrar se um operador tentar atribuir dados inválidos a uma viagem.
- **Critérios de Aceite**:
    - Crie uma exceção personalizada chamada `CapacidadeExcedidaException`.
    - Crie um método na classe `Veiculo` chamado `carregarItem(double pesoItem)`.
    - Se o peso do item ultrapassar a `capacidadeCarga` estipulada para o veículo, o sistema deve obrigatoriamente disparar (`throw`) a `CapacidadeExcedidaException`.
    - Na classe principal, utilize blocos `try-catch` para capturar o erro e exibir uma mensagem amigável ao usuário, impedindo o fechamento do programa.